

PruningContainer#

<https://pytorch.org/docs/stable/generated/torch.nn.utils.prune.PruningContainer.html>

Description:

Container holding a sequence of pruning methods for iterative pruning. Keeps track of the order in which pruning methods are applied and handles combining successive pruning calls. Accepts as argument an instance of a `BasePruningMethod` or an iterable of them. Add a child pruning method to the container. `method` (subclass of `BasePruningMethod`) – child pruning method to be added to the container.

Function Signature

```
class torch.nn.utils.prune.PruningContainer(*args)[source]#  
add_pruning_method(method)[source]#
```

Parameters

```
classmethod apply(module, name, *args,  
importance_scores=None, **kwargs)[source]#
```

Parameters

Returns:

pruned version of tensor t.

Notes & Warnings

NOTE

Note Pruning itself is NOT undone or reversed!

Detailed Documentation

Documentation

Container holding a sequence of pruning methods for iterative pruning.

Keeps track of the order in which pruning methods are applied and handles combining successive pruning calls.

Accepts as argument an instance of a `BasePruningMethod` or an iterable of them.

Add a child pruning method to the container.

method (subclass of `BasePruningMethod`) – child pruning method to be added to the container.

Add pruning on the fly and reparametrization of a tensor.

Adds the forward pre-hook that enables pruning on the fly and the reparametrization of

a tensor in terms of the original tensor and the pruning mask.

module (nn.Module) – module containing the tensor to prune

name (str) – parameter name within module on which pruning will act.

args – arguments passed on to a subclass of BasePruningMethod

importance_scores (torch.Tensor) – tensor of importance scores (of same shape as module parameter) used to compute mask for pruning. The values in this tensor indicate the importance of the corresponding elements in the parameter being pruned. If unspecified or None, the parameter will be used in its place.

kwargs – keyword arguments passed on to a subclass of a BasePruningMethod

Simply handles the multiplication between the parameter being pruned and the generated mask.

Fetches the mask and the original tensor from the module and returns the pruned version of the tensor.

module (nn.Module) – module containing the tensor to prune

pruned version of the input tensor

pruned_tensor (torch.Tensor)

Apply the latest method by computing the new partial masks and returning its combination with the default_mask.

The new partial mask should be computed on the entries or channels that were not zeroed out by the default_mask. Which portions of the tensor t the new mask will be calculated from depends on the PRUNING_TYPE (handled by the type handler):

for 'unstructured', the mask will be computed from the raveled list of nonmasked entries;

for 'structured', the mask will be computed from the nonmasked channels in the tensor;

for 'global', the mask will be computed across all entries.

t (torch.Tensor) – tensor representing the parameter to prune (of same dimensions as default_mask).

default_mask (torch.Tensor) – mask from previous pruning iteration.

new mask that combines the effects of the default_mask and the new mask from the current pruning method (of same dimensions as default_mask and t).

Compute and returns a pruned version of input tensor t.

According to the pruning rule specified in compute_mask().

t (torch.Tensor) – tensor to prune (of same dimensions as default_mask).

importance_scores (torch.Tensor) – tensor of importance scores (of same shape as t) used to compute mask for pruning t. The values in this tensor indicate the importance of the corresponding elements in the t that is being pruned. If unspecified or None, the tensor t will be used in its place.

default_mask (torch.Tensor, optional) – mask from previous pruning iteration, if any. To be considered when determining what portion of the tensor that pruning should act on. If None, default to a mask of ones.

pruned version of tensor t.

Remove the pruning reparameterization from a module.

The pruned parameter named `name` remains permanently pruned, and the parameter named `name+'_orig'` is removed from the parameter list. Similarly, the buffer named `name+'_mask'` is removed from the buffers.

Pruning itself is NOT undone or reversed!